JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class CopyOnWriteArraySet<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractSet<E>
      java.util.concurrent.CopyOnWriteArraySet<E>

Type Parameters:

`E - the type of elements held in this collection`

All Implemented Interfaces:

`Serializable, Iterable<E>, Collection<E>, Set<E>`

```
public class CopyOnWriteArraySet<E>
extends AbstractSet<E>
implements Serializable
```

A `Set` that uses an internal `CopyOnWriteArrayList` for all of its operations. Thus, it shares the same basic properties:

- It is best suited for applications in which set sizes generally stay small, read-only operations vastly outnumber mutative operations, and you need to prevent interference among threads during traversal.
- It is thread-safe.
- Mutative operations (`add`, `set`, `remove`, etc.) are expensive since they usually entail copying the entire underlying array.
- Iterators do not support the mutative `remove` operation.
- Traversal via iterators is fast and cannot encounter interference from other threads. Iterators rely on unchanging snapshots of the array at the time the iterators were constructed.

Sample Usage. The following code sketch uses a copy-on-write set to maintain a set of Handler objects that perform some action upon state updates.

```
 class Handler { void handle(); ... }
```

```
class X {
  private final CopyOnWriteArraySet<Handler> handlers
    = new CopyOnWriteArraySet<Handler>();
  public void addHandler(Handler h) { handlers.add(h); }

  private long internalState;
  private synchronized void changeState() { internalState = ...; }

  public void update() {
    changeState();
    for (Handler handler : handlers)
      handler.handle();
  }
}
```

This class is a member of the Java Collections Framework.

Since:

1.5

See Also:

CopyOnWriteArrayList, Serialized Form

Constructor Summary

Constructors

Constructor	Description
`CopyOnWriteArraySet()`	Creates an empty set.
`CopyOnWriteArraySet(Collection<? extends E> c)`	Creates a set containing all of the elements of the specified collection.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
boolean	`add(E e)`	Adds the specified element to this set if it is not already present.

boolean	**addAll**(**Collection**<? extends **E**> c)	Adds all of the elements in the specified collection to this set if they're not already present.
void	**clear**()	Removes all of the elements from this set.
boolean	**contains**(**Object** o)	Returns `true` if this set contains the specified element.
boolean	**containsAll**(**Collection**<? > c)	Returns `true` if this set contains all of the elements of the specified collection.
boolean	**equals**(**Object** o)	Compares the specified object with this set for equality.
void	**forEach**(**Consumer**<? super **E**> action)	Performs the given action for each element of the `Iterable` until all elements have been processed or the action throws an exception.
boolean	**isEmpty**()	Returns `true` if this set contains no elements.
Iterator<**E**>	**iterator**()	Returns an iterator over the elements contained in this set in the order in which these elements were added.
boolean	**remove**(**Object** o)	Removes the specified element from this set if it is present.
boolean	**removeAll**(**Collection**<? > c)	Removes from this set all of its elements that are contained in the specified collection.
boolean	**removeIf**(**Predicate**<? super **E**> filter)	Removes all of the elements of this collection

		that satisfy the given predicate.
boolean	**retainAll**(**Collection**<? > c)	Retains only the elements in this set that are contained in the specified collection.
int	**size**()	Returns the number of elements in this set.
Spliterator<**E**>	**spliterator**()	Returns a **Spliterator** over the elements in this set in the order in which these elements were added.
Object[]	**toArray**()	Returns an array containing all of the elements in this set.
<T> T[]	**toArray**(T[] a)	Returns an array containing all of the elements in this set; the runtime type of the returned array is that of the specified array.

Methods inherited from class java.util.AbstractSet

hashCode

Methods inherited from class java.util.AbstractCollection

toString

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.Collection

parallelStream, stream

Constructor Detail

CopyOnWriteArraySet

```
public CopyOnWriteArraySet()
```

Creates an empty set.

CopyOnWriteArraySet

```
public CopyOnWriteArraySet(Collection<? extends E> c)
```

Creates a set containing all of the elements of the specified collection.

Parameters:

`c - the collection of elements to initially contain`

Throws:

`NullPointerException - if the specified collection is null`

Method Detail

size

```
public int size()
```

Returns the number of elements in this set.

Specified by:

`size in interface Collection<E>`

Specified by:

`size in interface Set<E>`

Specified by:

`size in class AbstractCollection<E>`

Returns:

`the number of elements in this set`

isEmpty

```
public boolean isEmpty()
```

Returns `true` if this set contains no elements.

Specified by:

`isEmpty` in interface `Collection<E>`

Specified by:

`isEmpty` in interface `Set<E>`

Overrides:

`isEmpty` in class `AbstractCollection<E>`

Returns:

`true if this set contains no elements`

contains

```
public boolean contains(Object o)
```

Returns `true` if this set contains the specified element. More formally, returns `true` if and only if this set contains an element `e` such that `(o==null ? e==null : o.equals(e))`.

Specified by:

`contains` in interface `Collection<E>`

Specified by:

`contains` in interface `Set<E>`

Overrides:

`contains` in class `AbstractCollection<E>`

Parameters:

`o - element whose presence in this set is to be tested`

Returns:

`true if this set contains the specified element`

toArray

```
public Object[] toArray()
```

Returns an array containing all of the elements in this set. If this set makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order.

The returned array will be "safe" in that no references to it are maintained by this set. (In other words, this method must allocate a new array even if this set is backed by an array). The caller is thus free to modify the returned array.

This method acts as bridge between array-based and collection-based APIs.

Specified by:

toArray in interface Collection<E>

Specified by:

toArray in interface Set<E>

Overrides:

toArray in class AbstractCollection<E>

Returns:

an array containing all the elements in this set

toArray

```
public <T> T[] toArray(T[] a)
```

Returns an array containing all of the elements in this set; the runtime type of the returned array is that of the specified array. If the set fits in the specified array, it is returned therein. Otherwise, a new array is allocated with the runtime type of the specified array and the size of this set.

If this set fits in the specified array with room to spare (i.e., the array has more elements than this set), the element in the array immediately following the end of the set is set to `null`. (This is useful in determining the length of this set *only* if the caller knows that this set does not contain any null elements.)

If this set makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order.

Like the `toArray()` method, this method acts as bridge between array-based and collection-based APIs. Further, this method allows precise control over the runtime type of the output array, and may, under certain circumstances, be used to save allocation costs.

Suppose `x` is a set known to contain only strings. The following code can be used to dump the set into a newly allocated array of `String`:

```
  String[] y = x.toArray(new String[0]);
```

Note that `toArray(new Object[0])` is identical in function to `toArray()`.

Specified by:

toArray in interface Collection<E>

Specified by:

toArray in interface Set<E>

Overrides:

toArray in class AbstractCollection<E>

Type Parameters:

T - the runtime type of the array to contain the collection

Parameters:

`a - the array into which the elements of this set are to be stored, if it is big enough; otherwise, a new array of the same runtime type is allocated for this purpose.`

Returns:

`an array containing all the elements in this set`

Throws:

`ArrayStoreException - if the runtime type of the specified array is not a supertype of the runtime type of every element in this set`

`NullPointerException - if the specified array is null`

clear

```
public void clear()
```

Removes all of the elements from this set. The set will be empty after this call returns.

Specified by:

`clear in interface Collection<E>`

Specified by:

`clear in interface Set<E>`

Overrides:

`clear in class AbstractCollection<E>`

remove

```
public boolean remove(Object o)
```

Removes the specified element from this set if it is present. More formally, removes an element `e` such that `(o==null ? e==null : o.equals(e))`, if this set contains such an element. Returns `true` if this set contained the element (or equivalently, if this set changed as a result of the call). (This set will not contain the element once the call returns.)

Specified by:

`remove in interface Collection<E>`

Specified by:

`remove in interface Set<E>`

Overrides:

`remove in class AbstractCollection<E>`

Parameters:

o - object to be removed from this set, if present

Returns:

true if this set contained the specified element

add

public boolean add(E e)

Adds the specified element to this set if it is not already present. More formally, adds the specified element e to this set if the set contains no element e2 such that (e==null ? e2==null : e.equals(e2)). If this set already contains the element, the call leaves the set unchanged and returns false.

Specified by:

add in interface Collection<E>

Specified by:

add in interface Set<E>

Overrides:

add in class AbstractCollection<E>

Parameters:

e - element to be added to this set

Returns:

true if this set did not already contain the specified element

containsAll

public boolean containsAll(Collection<?> c)

Returns true if this set contains all of the elements of the specified collection. If the specified collection is also a set, this method returns true if it is a *subset* of this set.

Specified by:

containsAll in interface Collection<E>

Specified by:

containsAll in interface Set<E>

Overrides:

containsAll in class AbstractCollection<E>

Parameters:

c - collection to be checked for containment in this set

Returns:

`true` if this set contains all of the elements of the specified collection

Throws:

`NullPointerException` - if the specified collection is null

See Also:

`contains(Object)`

addAll

```
public boolean addAll(Collection<? extends E> c)
```

Adds all of the elements in the specified collection to this set if they're not already present. If the specified collection is also a set, the `addAll` operation effectively modifies this set so that its value is the *union* of the two sets. The behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Specified by:

`addAll` in interface `Collection<E>`

Specified by:

`addAll` in interface `Set<E>`

Overrides:

`addAll` in class `AbstractCollection<E>`

Parameters:

`c` - collection containing elements to be added to this set

Returns:

`true` if this set changed as a result of the call

Throws:

`NullPointerException` - if the specified collection is null

See Also:

`add(Object)`

removeAll

```
public boolean removeAll(Collection<?> c)
```

Removes from this set all of its elements that are contained in the specified collection. If the specified collection is also a set, this operation effectively modifies this set so that its value is the *asymmetric set difference* of the two sets.

Specified by:

removeAll in interface Collection<E>

Specified by:

removeAll in interface Set<E>

Overrides:

removeAll in class AbstractSet<E>

Parameters:

c - collection containing elements to be removed from this set

Returns:

true if this set changed as a result of the call

Throws:

ClassCastException - if the class of an element of this set is incompatible with the specified collection (optional)

NullPointerException - if this set contains a null element and the specified collection does not permit null elements (optional), or if the specified collection is null

See Also:

remove(Object)

retainAll

public boolean retainAll(Collection<?> c)

Retains only the elements in this set that are contained in the specified collection. In other words, removes from this set all of its elements that are not contained in the specified collection. If the specified collection is also a set, this operation effectively modifies this set so that its value is the *intersection* of the two sets.

Specified by:

retainAll in interface Collection<E>

Specified by:

retainAll in interface Set<E>

Overrides:

retainAll in class AbstractCollection<E>

Parameters:

c - collection containing elements to be retained in this set

Returns:

true if this set changed as a result of the call

Throws:

ClassCastException - if the class of an element of this set is incompatible with the specified collection (optional)

NullPointerException - if this set contains a null element and the specified collection does not permit null elements (optional), or if the specified collection is null

See Also:

remove(Object)

iterator

public Iterator<E> iterator()

Returns an iterator over the elements contained in this set in the order in which these elements were added.

The returned iterator provides a snapshot of the state of the set when the iterator was constructed. No synchronization is needed while traversing the iterator. The iterator does *NOT* support the remove method.

Specified by:

iterator in interface Iterable<E>

Specified by:

iterator in interface Collection<E>

Specified by:

iterator in interface Set<E>

Specified by:

iterator in class AbstractCollection<E>

Returns:

an iterator over the elements in this set

equals

public boolean equals(Object o)

Compares the specified object with this set for equality. Returns true if the specified object is the same object as this object, or if it is also a Set and the elements returned by an iterator over the specified set are the same as the elements returned by an iterator over this set. More formally, the two iterators are considered to return the same elements if they return the same number of elements and for every element e1 returned by the iterator over the specified set, there is an element e2 returned by the iterator over this set such that (e1==null ? e2==null : e1.equals(e2)).

Specified by:

equals in interface Collection<E>

Specified by:

equals in interface Set<E>

Overrides:

equals in class AbstractSet<E>

Parameters:

o - object to be compared for equality with this set

Returns:

true if the specified object is equal to this set

See Also:

Object.hashCode(), HashMap

removeIf

```
public boolean removeIf(Predicate<? super E> filter)
```

Description copied from interface: Collection

Removes all of the elements of this collection that satisfy the given predicate. Errors or runtime exceptions thrown during iteration or by the predicate are relayed to the caller.

Specified by:

removeIf in interface Collection<E>

Parameters:

filter - a predicate which returns true for elements to be removed

Returns:

true if any elements were removed

forEach

```
public void forEach(Consumer<? super E> action)
```

Description copied from interface: Iterable

Performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception. Unless otherwise specified by the implementing class, actions are performed in the order of iteration (if an iteration order is specified). Exceptions thrown by the action are relayed to the caller.

Specified by:

forEach in interface Iterable<E>

Parameters:

action - The action to be performed for each element

spliterator

```
public Spliterator<E> spliterator()
```

Returns a `Spliterator` over the elements in this set in the order in which these elements were added.

The `Spliterator` reports `Spliterator.IMMUTABLE`, `Spliterator.DISTINCT`, `Spliterator.SIZED`, and `Spliterator.SUBSIZED`.

The spliterator provides a snapshot of the state of the set when the spliterator was constructed. No synchronization is needed while operating on the spliterator.

Specified by:

`spliterator` in interface `Iterable<E>`

Specified by:

`spliterator` in interface `Collection<E>`

Specified by:

`spliterator` in interface `Set<E>`

Returns:

a Spliterator over the elements in this set

Since:

1.8

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.